

EC-GCN: A encrypted traffic classification framework based on multi-scale graph convolution networks

Zulong Diao^{a,b}, Gaogang Xie^c, Xin Wang^f, Rui Ren^{a,d}, Xuying Meng^{a,b}, Guangxing Zhang^{a,*}, Kun Xie^e, Mingyu Qiao^{a,d}

^a ICT/CAS, Chinese Academy of Sciences, China

^b Purple Mountain Laboratories, China

^c CNIC/CAS, Chinese Academy of Sciences, China

^d University of Chinese Academy of Sciences, China

^e Computer Science and Electronics Engineering, Hunan University, China

^f Electrical and Computer Engineering, State University of New York at Stony Brook, USA

ARTICLE INFO

Keywords:

Encrypted traffic classification

Graph neural networks

Packet sequence learning

ABSTRACT

The sharp increase in encrypted traffic brings a huge challenge to traditional traffic classification methods. Combining deep learning with time series analysis techniques is a recent trend in solving this problem. Most of these approaches only capture the temporal correlation within a flow. The accuracy and robustness are unsatisfactory, especially in an unstable network environment with high packet loss and reordering. How to learn a representation with a strong generalization ability for each encrypted traffic flow remains a key challenge. Our detailed analysis indicates that there is a graph with particular local structures corresponding to each type of encrypted traffic flow. Inspired by this observation, we propose a novel deep learning framework called EC-GCN to classify encrypted traffic flows based on multi-scale graph convolutional neural networks. We first provide a novel lightweight layer that only relies on the metadata and encodes each encrypted traffic flow into graph representations. So that our framework can be independent of different encryption protocols. Then we design a novel graph pooling and structure learning layer to dynamically extract the multi-graph representations and improve the capabilities to adapt to complex network environments. EC-GCN is an end-to-end classification model that learns representative spatial-temporal traffic features hidden in a traffic time series and then classifies them in a unified framework. Our comprehensive experiments on three real-world datasets indicate that EC-GCN can achieve up to 5%–20% accuracy improvement and outperforms state-of-the-art methods.

1. Introduction

In recent years, network traffic classification has become possible due to breakthroughs in technology as well as the speedy growth of high-speed internet traffic demands. The accurate classification of network traffic can help manage and secure network resources, and also plays a vital role in Quality of Service (QoS) provisioning, billing in ISPs to security-related applications in firewalls, and providing differentiated services. The process of traffic classification often includes capturing the network traffic on a target network link, building behavioral fingerprints and then identifying the traffic according to the classification criterion. However, widely used encryption techniques (IPSec, TLS/SSL etc.) randomize all the communication contents, which

makes the performance of traditional traffic classification methods suffer. Therefore, how classifying the encrypted traffic accurately attracts widespread attention from both industries and academia.

In order to improve the classification accuracy of encrypted traffic, some methods pay attention to fields in packet headers, statistic characteristics or contextual information of a network flow. In the packet header, cipher suites and digital certificate features have been proven in many studies to be informative for traffic fingerprints. DNS-based algorithms can immediately classify network flows, requiring just the IP header of the first packet and the information extracted from DNS query-response conversations. However, with the rapid evolution of network protocols over time, some previously unencrypted fields began to be encrypted, reducing the classification performance of these

* Corresponding author.

E-mail addresses: diaozulong@ict.ac.cn (Z. Diao), xie@cnic.cn (G. Xie), x.wang@stonybrook.edu (X. Wang), renrui1234@gmail.com (R. Ren), mengxuying@ict.ac.cn (X. Meng), guangxing@ict.ac.cn (G. Zhang), xiekun@hnu.edu.cn (K. Xie), qiaomingyudinghan@163.com (M. Qiao).

<https://doi.org/10.1016/j.comnet.2023.109614>

Received 25 August 2022; Received in revised form 26 January 2023; Accepted 3 February 2023

Available online 10 February 2023

1389-1286/© 2023 Elsevier B.V. All rights reserved.

methods (DNS to DOH, TLS1.2 to TLS1.3). Some end-to-end models, based on deep learning algorithms and time-series features, emerge. These models do not rely on data packet fields, which allows them to be protocol-independent and handle both encrypted and unencrypted traffic. An end-to-end model can combine feature engineering and model training into a unified model and learn features from the raw input directly. The learned features are guided by real labels to boost performance. However, the available information for the end-to-end models is limited compared with previous methods. Most of them only extract the time correlation within a time series, which leads to low classification accuracy and adaptability. As traffic distribution varies at different network locations, the performance of these approaches is also different. In addition, The real-time packet length sequence of encrypted traffic data may be partially abnormal (packet loss/packet reordering) as a result of network failures. The multiple aggregations of traffic flows will also have a great impact on the performance of the model due to the change in the time interval between consecutive packets.

In light of the issues above, we propose an adaptive end-to-end model, named EC-GCN. Compared with existing encrypted traffic classifying methods, our paper makes the following contributions:

- We creatively represent an encrypted traffic flow as a graph and we observe many obvious characteristics which reveal potential spatial patterns in the encrypted traffic.
- To learn the spatial dependence hidden in the traffic flow, we creatively introduce GCN into our classification method and propose a novel temporal-spatial (multi-subgraph) encrypted traffic classification framework. To the best of our knowledge, this is the first multi-graph-based method that is applied to classify the encrypted traffic. To make our method robust to noise and dynamics, we organize the graph into multiple granularity levels and exploit the features from all levels.
- In the deep learning framework, we creatively design an encoding layer to automatically convert an encrypted traffic flow to a graph representation. In addition, we propose a novel graph-pooling layer and graph-learning layer to dynamically extract the multi-graph structures in an encrypted traffic flow.
- To evaluate the performance of our proposed model, we conduct a series of experiments across 3 datasets. The experiment results show that EC-GCN can achieve up to 5%–20% precision improvement in these experiments and is more fault-tolerant compared with the state-of-the-art methods.

The rest of this paper is organized as follows. We first summarize the related work of encrypted traffic classification in Section 2, and provide some data analysis results and motivations in Section 3. We then present the technical details of our novel EC-GCN model in Section 4. After that, we evaluate the performance of our proposed model through experiments on 3 data sets in Section 5, and conclude our work in Section 6.

2. Related works

Encrypted traffic analysis has attracted intensive attention. According to the methodologies used, we divide the previous research into several categories to review.

2.1. Conventional traffic classification

Some studies finish the traffic classification task by analyzing the used communication ports contained in the TCP/UDP header. The well-known ports for the protocols are assigned by the IANA [1]. However, this method is failed in situations with port dynamic allocation [2] or masquerading ports. In addition, this approach also fails on tunnels or Network Address Port Translation (NAPT).

Payload-based method [3–5] uses the specific signature strings in the payload for matching, such as the Deep Packet Inspection (DPI) approaches [6]. Keralapura et al. provided a self-learning traffic classifier to identify the P2P traffic in high-speed networks with application payload signatures [7]. The payload-signature-based classifier is considered a reliable method for Internet traffic classification but is prohibitively and computationally expensive for the real-time handling of large amounts of traffic on a high-speed network. To solve this problem, most studies focus on the pattern matching algorithm or hardware-based approaches using FPGA or network processors. Yang et al. [4] propose a novel multi-stride regular expression matching engine based on FPGA to solve the memory explosion problem of multi-stride DPI algorithms. Doroud et al. [5] proposed a multi-stage and modular architecture to combine state-of-the-art classifiers in order to improve the overall efficiency of DPI, while maintaining an equivalent classification accuracy. However, as the application payload has been randomized by various encryption techniques, payload-based methods often lose their efficiency in classifying the encrypted traffic if not completely infeasible.

2.2. Fingerprint construction

As conventional traffic classification may fail under encrypted traffic, some studies [8–11] suggest using unencrypted protocol field information, typically layer 3 and layer 4 information to represent each flow and constructs a fingerprint library by clustering and cross-correlating for efficient traffic classification. Yoon et al. [12] propose a novel fingerprint maintenance method using the properties of identified traffic and the usage history of the signatures. Lee et al. [13] propose an automatic header-signature naming system and identification system using the named header-signature. The proposed system provides efficient management of the header signature of each service as well. The main problem of the fingerprint-based identification method is the unstable classification performance and a large-scale fingerprint library that need to be maintained.

2.3. Statistical and traditional machine learning approach

Statistical Features are proposed to solve encrypted traffic classification problems together with various traditional machine learning algorithms [14,15]. For instance, Anderson et al. took flow-level features and unencrypted TLS header information as joint features to identify malware encrypted traffic with the logistic regression algorithm [16]. A robust application identification method with the concept of the burst and flow statistical features was proposed by work [17]. In order to reduce the impact of the high dimensionality and redundancy of flow statistical features, the imbalance in the number of traffic flows and the concept drift of Internet traffic on classification performance, And Shi et al. propose a new feature optimization approach based on deep learning and Feature Selection (FS) techniques to provide the optimal and robust features for traffic classification [18]. However, these methods are mainly based on rich experiences, professional knowledge and lots of human effort.

Some studies apply Markov models for learning the generation probabilities of features and identifying the corresponding application [19–21]. Chang et al. propose a multi-attribute encrypted traffic classification method that integrates both message type sequences and packet length sequences to build Markov models [22]. Moreover, Fu et al. develop a system for classifying service usages of mobile messaging Apps by jointly modeling user behavioral patterns, network traffic characteristics, and temporal dependencies [23]. However, these

methods cannot handle the long-term relationship due to the small order (e.g., 1 or 2) of Markov model.

2.4. Encrypted traffic classification based on deep learning

With the evolution of network technology and protocols, some previously unencrypted fields in the packet header are now encrypted, such as DOH, TLS 1.3 etc. These changes cause many encrypted classification methods to fail. With the advantages of automatic feature extraction and end-to-end classification, some studies attempt to apply Deep learning (DL) for encrypted traffic classification. DLWF et. [24–26] exploit convolution neural network(CNN) and long short-term memory(LSTM) to propagate the flow and packet-based features. FS-Net [27] uses an end-to-end encrypted traffic classification model which includes an encoder to generate the features, and a decoder with a reconstruction layer to restore the input sequences. Deeppacket [28] adopts the stacked auto-encoder and one-dimensional convolution neural network to extract features from encrypted traffic payloads automatically.

There are also some researchers who pay attention to sequential metadata features and propose protocol-independent classification methods based on DL [29,30]. Common sequential metadata include packet length sequences, packet type sequences, packet interval sequences and uplink/downlink sequences etc. Traffic classification based on metadata helps reduce the impact of encryption. However, the previous research only utilizes the time correlation in the metadata sequence, resulting in the actual performance in many network environments being unsatisfactory. In this paper, we design a novel graph-based deep learning method that captures the temporal and spatial dependence to enhance the performances of both the feature representation and classification.

3. Analysis and motivations

Before presenting our detailed design of the classifying framework based on GCN, we provide some data analysis results and learn the characteristics of the graph in each traffic flow. The datasets, which we will describe in detail in Section 5.1 are collected by us from a real network environment. The analysis results in the three datasets are similar. Due to space limitations, we only present the analysis results based on the first dataset (30 types of HTTPS web applications). Different from the temporal pattern learning in packet length sequences, we regard each network flow as a graph and observe their differences.

3.1. Graph structure analysis

In order to visualize the graph representation in a network traffic flow, we define a node in the graph as a state of packet length (from 0 to the maximum transmission unit MTU) and an edge as the association between two nodes (directly related to the transition probability between the two corresponding packet lengths in the same traffic flow). We select the first M consecutive packets from each traffic flow to form a time series. The specific value of the parameter M will be discussed in Section 5. We first calculate the transition matrix $TM \in \mathbb{R}^{MTU \times MTU}$ based on the packet length series for each traffic flow. Each item in TM_{ij} represents the transition number from the node i to the node j . Then in order to analyze the graph structure of different web applications, we directly sum up all the transition matrices of the same applications. We delete an edge if its corresponding item in the transition matrix is less than a certain threshold. As shown in Fig. 1, different graph numbers (from 1 to 30) represent different web applications and different node colors in a graph represent different packet length intervals. The larger the weighted degree, the larger the frequency that the corresponding graph node (i.e., the specific packet

length) appears in the flow. It can be seen from Fig. 1 that there are obvious characteristics:

- Different web applications show different graph topologies, such as star topology in the graph 3/9/13/14, dual-core topology in the graph 1/8, multi-subgraph topology in the graph 6. This phenomenon shows that there are obvious differences in the communication modes of different web applications.
- Different web applications have different core graph nodes, such as 54/1394 in the graph 1, 74/1494 in the graph 5, 66/1514 in the graph 9. This phenomenon shows that the graph structure can reveal the ACK mechanism of different web applications to some extent.
- Similar web applications have some common features, such as the similar diamond structure in the graph 12/15 which correspond to Weixin/Aiqiyi with the same operating system OSX and the same web browser Chrome.

3.2. Graph node and edge analysis

As packet lengths may vary from 0 to MTU, if we map each possible packet length to a graph node in a traffic flow, then the graph may include thousands of nodes at least. The execution complexity of a GCN-based model is directly related to the scale of the graph. With the number of graph nodes increasing, the execution time would greatly increase which will be conducive to the online traffic classification tasks. Therefore, we also carry out some graph node analysis work in this part and explore the feasibility of introducing GCN into our traffic classification tasks.

At first, we draw the packet length distribution of all web applications. As shown in Fig. 2(a), the packet lengths of different applications have long-tail distribution and a small amount of packet length covers most of the proportion. This phenomenon shows that the number of actual graph nodes corresponding to a traffic flow is limited. In addition, as shown in Fig. 2(b), we analyze the average node degree in a graph (corresponding to a traffic flow) of different applications. We find that the average node degree in a graph is between 3.60 and 5.82. This phenomenon shows that the number of actual graph edges corresponding to a traffic flow is limited. Above all, we can conclude that **introducing GCN into our online traffic classification tasks with a limited-scale graph structure is feasible.**

4. Problem and model

4.1. Problem definition

The encrypted traffic classification problem in this paper is to classify the encrypted traffic into specific applications with the packet length sequences as inputs. As all the communication contents are randomized after encryption, we do not plan to use any content from the packet payload or header. Thus the only information we can use is the metadata which can be represented as a time series. The metadata includes packet lengths, packet directions and the inter-arrival time. Considering the extremely unstable characteristics of the inter-arrival time as it is often affected by the network environment, we only choose the first two. Assume that there are T applications in total. Let the sequence of a sample be $(x_1, x_2, \dots, x_M)$, where M corresponds to how many packets we will choose from each traffic flow to form a time series and x_i is the i th packet length. The associated label can be represented as a vector $Y \in \mathbb{R}^T$, i.e., if the sample belongs to class j , then $Y(j) = 1$, otherwise $Y(j) = 0$. We aim to build an end-to-end model to predict a label $\hat{Y}$ that can accurately classify all applications.

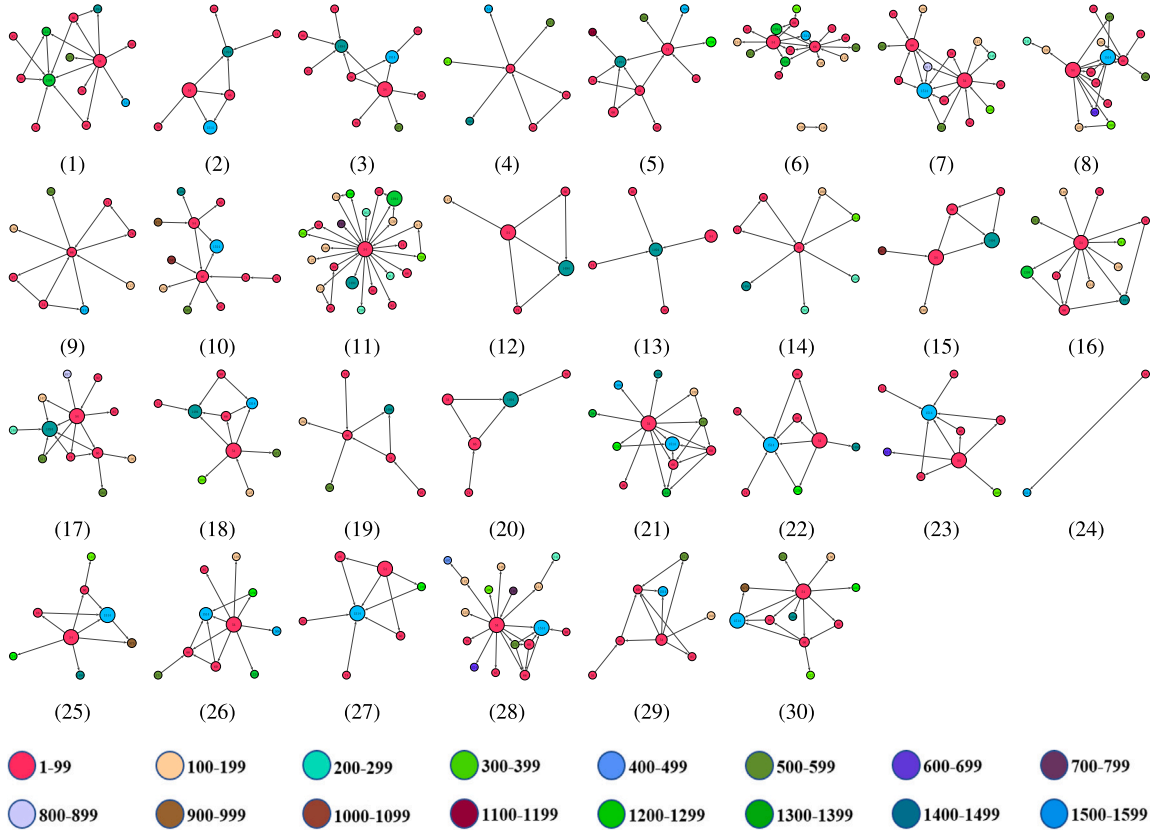


Fig. 1. Graph structures of different applications. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

4.2. The overall neural network architecture

Only relying on the interflow metadata makes our method independent of encryption protocol details. Through previous analysis, we find that the packet length sequence has both temporal (packet length varies with transmission order) and spatial correlation (graph representation). Exploiting GCN drives us to cope with the encrypted traffic classification tasks from a high-dimensional perspective. However, when designing a novel graph-based classification model, we will still face tough challenges and need to find solutions to the following questions:

- Q1: How to achieve performance improvement with so limited information provided from the interflow metadata?
- Q2: How to encode the traffic time series to a graph representation for GCN and how to control the scale of the graph so as not to affect the efficiency of online classification?

We convert the encrypted traffic identification problem into a graph classification problem. Fig. 4 provides an overview of our proposed framework EC-GCN. In order to resolve the first question, we attempt to construct multi-level graph representations based on GCN and the interflow metadata of each flow, which enables us to observe the behavior of different encrypted network flows from multiple dimensions and multiple scales. In addition, we novelly propose a one-hot layer, a lightweight graph pooling layer and a structure learning layer to reduce the complexity of graph representation learning. The proposed framework is composed of two major components: (1) 6 temporal blocks to capture temporal correlation and form low-dimensional features for each node in a graph. (2) 6-level of spatial blocks, which capture the graph representations at different levels and then summarize them in a hierarchical way. To extract more general features from the aggregate of packets, we apply the graph pooling layer to scale down the size of inputs by grouping similar nodes into the same cluster to enlarge the receptive fields in a graph. We also propose a novel structure learning

layer to learn a refined graph structure in the pooled graph. Between temporal blocks and spatial blocks, we add an FC layer to merge the temporal features of all channels into unified features for each node.

4.3. Building a encrypted traffic flow into a graph

In this section, we first present the detailed definition of a graph in the encrypted traffic flows. We use $g = (v, \epsilon, X)$ to represent a graph where v denotes the set of $|v| = N$ graph nodes, ϵ denotes a set of edges and W denotes the weight matrix of G .

- Graph Node: We divide the packet length range from 0 to MTU into N intervals on average. Each interval represents a graph node.
- Edge: The corresponding correlation between two graph nodes

When constructing the graph structure, the traditional packet header fields such as source-Ip and destination-Ip are not used. We attempt to finish the classification process by relying solely on traffic metadata sequences (such as packet length sequences). So that we can overcome the distractions of traffic encryption and reduce the complexity of online traffic feature extraction.

Now, we present how we transform the packet length sequence of an encrypted traffic flow into a graph representation. Given a traffic flow with M packets ($M = 1000$) and the number of intervals N we divide is 40, as shown in Fig. 3, we creatively use the one-hot encoding process of N digits to transform the packet length sequence into a graph representation. Take the first packet of size 1520 as an example, its interval number is 40. So that the 40-th digit at the first column will be set to 1. One-hot encoding is a sparse way of representing data in a binary string in which only a single bit can be 1, while all others are 0. By introducing the one-hot encoding, we can convert the sequence data ($\mathbb{R}^M$) into the graph representation ($\mathbb{R}^{N \times M}$) and use it as input to

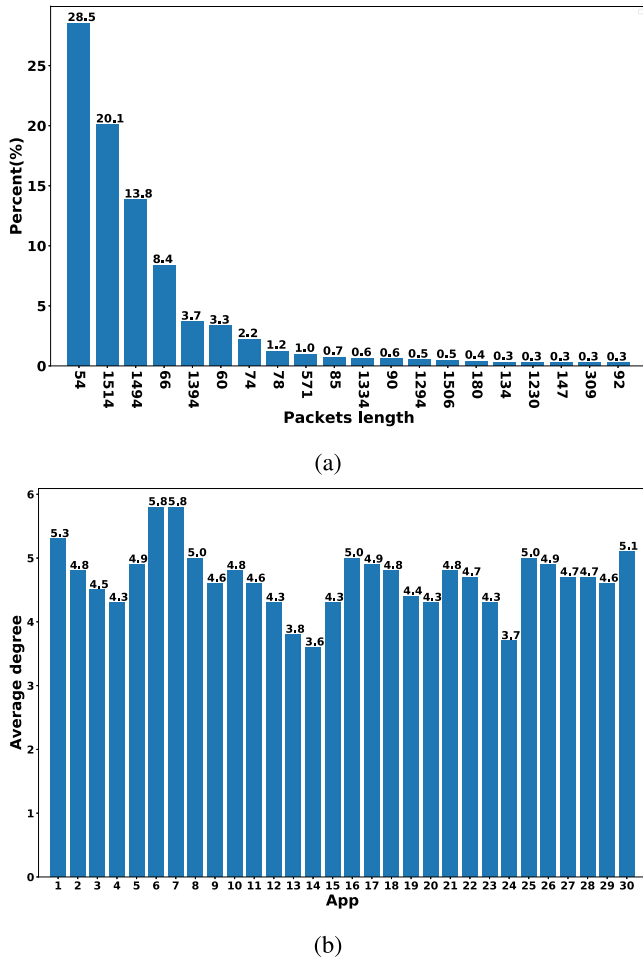


Fig. 2. The packet length and degree distribution of all web applications.

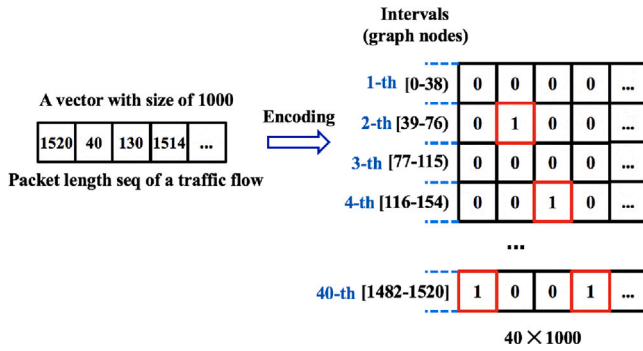


Fig. 3. The encoding process in which we transform the sequence data into a graph representation.

the spatiotemporal model, where N, M corresponding to the number of graph nodes in the spatial domain and the numbers of packets to consider in the temporal domain. In addition, the sparse representation may help us speed up our deep learning framework further.

We can sum up the graph representation ($\mathbb{R}^{N \times M}$) of all traffic flows (in the training set) and calculate its corresponding covariance matrix as the initial weight matrix of G . Taking 40 intervals as an example, as shown in Fig. 5, there are obvious differences in the feature similarity between different graph nodes. The higher the similarity between the two intervals, the more likely the corresponding packet sizes may appear adjacent to each other in the traffic flow. Apart from

the common initial weight matrix, we also provide the method for dynamically learning the weight matrices under different levels of each specific traffic flow, which we will describe in detail in Section 4.7.

4.4. The temporal block

After going through the one-hot layer, we obtain a sparse but linearly independent representation ($\mathbb{R}^{N \times M \times 1}$) in the spatial dimension direction, where N, M correspond to the number of nodes in the spatial domain and numbers of packets to consider in the temporal domain. In order to compress the temporal features and capture the spatial correlation between different nodes in the graph, we design 6 temporal blocks which can take full advantage of temporal dependencies hidden in traffic data. Each temporal block includes one 1-D temporal convolutional layer and one max-pooling layer to achieve the translation invariant and downsampled feature maps. As the max-pooling output will not change when there is a translation of packets, the result will not be sensitive to the local packet reordering. Every 2 temporal blocks will be followed by a batch-normalization layer to maintain the consistency of data distribution. Through 6 temporal blocks, the input will be converted to the hidden representation $H^{(l)} \in \mathbb{R}^{N \times m^l \times c^l}$, where N, m^l, c^l corresponding to the number of nodes in the spatial domain, the size of temporal dimension and the number of feature maps in the l th temporal block. Then we attach a fully connected layer (FC) which is responsible for merging the temporal compressed features from all channels into F unified spatial features. The entire temporal blocks transfer the input ($\mathbb{R}^{N \times M \times 1}$) to the initial node embeddings ($\mathbb{R}^{N \times F}$). Given that recurrent networks for traffic classification may suffer from time-consuming iterations/training and complex gate mechanisms, we employ entire convolutional structures on the time axis to capture the temporal dynamic behaviors of encrypted traffic flows.

4.5. Graph convolutional neural network

We introduce GCN [31] into our model and briefly review its mechanism in this subsection. Given a directed graph $G = (v, \epsilon, W)$, where the set v contains $|v| = N$ vertices; ϵ represents a set of edges, W denotes the weight matrix of G . We have the graph Laplacian $L = D - W$ and the eigendecomposition of $L = U\Lambda U^T$, where D is a diagonal matrix with the i_{th} element of the diagonal line being the degree of the node i : $D_{ii} = \sum_j W_{ij}$ and U is an orthogonal matrix.

A signal on graph G of N nodes can be described as a matrix X consisting of C_{in} vectors of size N . Consequently, for the signal $X = [x_1, x_2, \dots, x_{C_{in}}]$, a 1-D graph convolution operation with a kernel tensor θ of size (c_{in}, c_{out}, K) is

$$y_j = \sum_{i \in [1, c_{in}], k \in [1, K]} \theta_{ijk} L^k x_i, j = 1, 2, \dots, c_{out} \quad (1)$$

where C_{in} and C_{out} represent the size of the input feature map and the output feature map respectively.

4.6. The lightweight graph pooling layer

In this subsection, we introduce our proposed graph pooling operation to enable the down-sampling of the graph data. Compared with traditional graph classification tasks, the graph representations in our methods are relatively fixed which is mainly manifested in the following areas: (1) The number of graph nodes is fixed and corresponds to the number of intervals divided according to the range of packet length (0-MTU); (2) Each graph node corresponds to a fixed packet length interval; (3) The order of graph nodes in the initial graph representation $H \in \mathbb{R}^{N \times F}$ is fixed (Arranged by the corresponding packet length in ascending order).

Online traffic classification tasks have high requirements for execution time. The relatively fixed graph nodes drive us to adopt a more lightweight pooling operation. The pooling operation identifies a subset

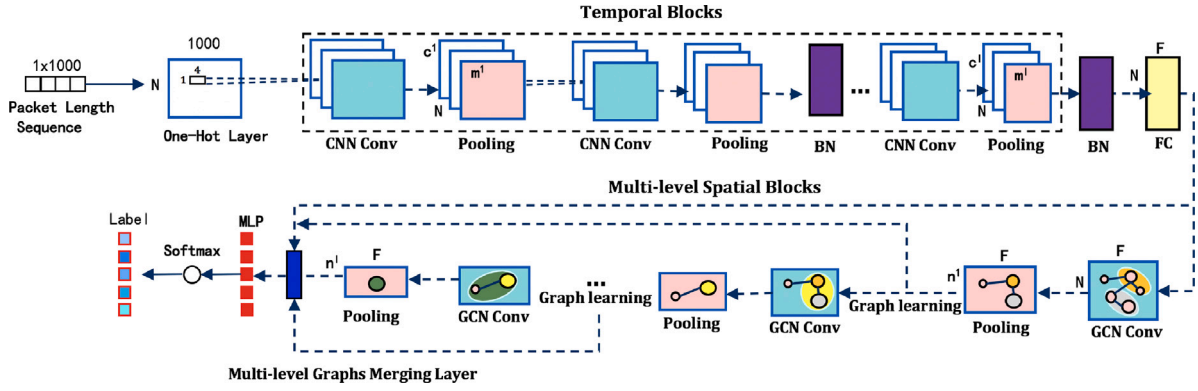


Fig. 4. Architecture of proposed EC-GCN framework combined.

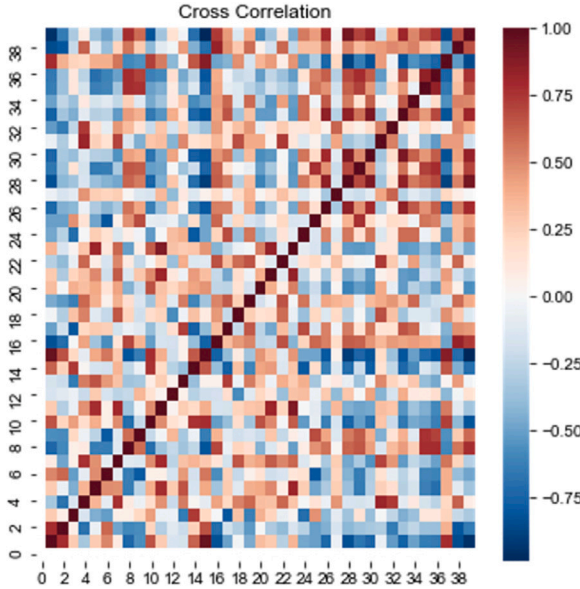


Fig. 5. The spatial correlation between different intervals.

of informative nodes to form a new but smaller graph. We denote the learned cluster assignment matrix at the layer l as $S^{(l)} \in \mathbb{R}^{n^l \times n^{l+1}}$. Each row of $S^{(l)}$ corresponds to one of the n^l nodes (or clusters) at layer l , and each column of $S^{(l)}$ corresponds to one of the n^{l+1} clusters at the next layer $l+1$. Intuitively, $S^{(l)}$ provides a soft assignment of each node at layer l to a cluster in the next coarsened layer $l+1$. We denote the weight matrix at this layer as $W^{(l)}$ and denote the node embedding matrix at this layer as $H^{(l)}$ (The initial process of the weight matrix is presented in Section 4.3). Given these inputs, the POOL layer coarsens the input graph, generating a new coarsened weight matrix $W^{(l+1)}$ and a new matrix of embeddings $H^{(l+1)}$ for each of the nodes/clusters in this coarsened graph. In order to keep low complexity, different from [32], the assignment matrix $S^{(l)} \in \mathbb{R}^{n^l \times n^{l+1}}$ is directly set as a parameter learned during the training procedure. In addition, we replace the adjacency matrix with the weighted adjacency matrix to learn more connectivity information between different clusters. In particular, we apply the two following equations:

$$\begin{aligned} H^{(l+1)} &= S^{(l)T} H^{(l)} \in \mathbb{R}^{n^{l+1} \times F} \\ W^{(l+1)} &= S^{(l)T} W^{(l)} S^{(l)} \in \mathbb{R}^{n^{l+1} \times n^{l+1}} \end{aligned} \quad (2)$$

Eq. (2) takes the node embeddings $H^{(l)}$ and aggregates these embeddings according to the cluster assignments $S^{(l)}$, generating embeddings for each of the n^{l+1} clusters. Once the training process finishes, these

assignment matrices $S^{(l)} \in \mathbb{R}^{n^l \times n^{l+1}}$ between different layers are fixed. That is, the subgraph nodes at different levels of encrypted traffic are also fixed. Our method aims to extract the exact fingerprint from different types of encrypted traffic flow according to the graph representations at different levels. Fixed nodes do not mean fixed graph representations. On the contrary, fixed nodes at different subgraphs can help simplify traffic classification problems and restrict the range of traffic fingerprints.

4.7. The structure learning layer

Our structure learning layer aims to dynamically learn the weighted adjacency matrix at different levels and thus adaptively generate graph representations with the common graph nodes according to different traffic flows. The pooling operation might lead highly related nodes to be disconnected in the induced subgraph, which loses the completeness of the graph structure information and further hinders the message passing procedure. Here, we design a structure learning algorithm, which can fully utilize both the node features and graph structure information.

We first define an $IR = D^{(l)-1} W^{(l)} H^{(l)}$, $IR \in \mathbb{R}^{n^l \times F}$ matrix to represent the interactive behavior of graph nodes. Each row of IR corresponds to the weighted average of feature vectors of all neighbor nodes of the specified node. $W^{(l)} \in \mathbb{R}^{n^l \times n^l}$ is the weight adjacency matrix in which diagonal elements are zero. $D^{(l)}$ represents the diagonal degree matrix of $W^{(l)}$. The matrix $IR \in \mathbb{R}^{n^l \times F}$ encodes the interactive score of each node in the graph. The larger the values of items in a row of IR , the more frequent interactions a node with its neighbor nodes.

Algorithm 1 The Structure Learning Algorithm

Input: input $H^{(l)} \in \mathbb{R}^{n^l \times F}$, weight matrix $W \in \mathbb{R}^{n^l \times n^l}$
Output: updated weight matrix $W' \in \mathbb{R}^{n^l \times n^l}$

- 1: Normalize $H^{(l)} = \frac{H^{(l)} - \bar{H}^{(l)}}{\sigma}$, where $\bar{H}^{(l)}$ and σ represent the corresponding mean and standard deviation matrix.
- 2: Compute the matrix $IR = D^{(l)-1} W^{(l)} H^{(l)}$
- 3: Define a parameter $P \in \mathbb{R}^{1 \times F}$
- 4: Duplicate the parameter P n^l times along the first dimension ($P \in \mathbb{R}^{n^l \times F}$)
- 5: Update the weight matrix $W' = Relu(\sqrt{|IR \odot P * IR^T|})$
- 6: Normalize W' and Set the diagonal of W' to zeros
- 7: Return W'

Different from traditional methods, we choose to define the similarity between two nodes from a global perspective, rather than just based on the similarity of features between two nodes. If the interactive behavior of two nodes is similar, then the two nodes are similar. This can effectively avoid the impact of noise or data offset on classification

Table 1

The 30 applications on OBW30 dataset.

ID	Apps	ID	Apps
1	osx+chrome+baidu	16	ubuntu+firefox+baidu
2	osx+chrome+iqiyi	17	ubuntu+firefox+iqiyi
3	osx+chrome+taobao	18	ubuntu+firefox+taobao
4	osx+chrome+weibo	19	ubuntu+firefox+weibo
5	osx+chrome+weixin	20	ubuntu+firefox+weixin
6	osx+firefox+baidu	21	windows+chrome+baidu
7	osx+firefox+iqiyi	22	windows+chrome+iqiyi
8	osx+firefox+taobao	23	windows+chrome+taobao
9	osx+firefox+weibo	24	windows+chrome+weibo
10	osx+firefox+weixin	25	windows+chrome+weixin
11	ubuntu+chrome+baidu	26	windows+firefox+baidu
12	ubuntu+chrome+iqiyi	27	windows+firefox+iqiyi
13	ubuntu+chrome+taobao	28	windows+firefox+taobao
14	ubuntu+chrome+weibo	29	windows+firefox+weibo
15	ubuntu+chrome+weixin	30	windows+firefox+weixin

Table 2

The 19 applications on HW19 dataset.

ID	Apps	ID	Apps
1	Alipay	11	Taobao
2	Baidu	12	Weibo
3	Github	13	Zhihu
4	iCloud	14	Toutiao
5	JD	15	Douban
6	Mozilla	16	TikTok
7	NeCmusic	17	Douyu
8	Youdao	18	Bilibili
9	QQ	19	Pinduoduo
10	Sogou		

performance. The detailed learning process is shown in Algorithm 1. We first normalize the hidden input $H^{(l)}$ to reduce the deviation of the data distribution caused by the previous convolution process. Then we compute the matrix IR to represent the interactive behavior of each graph node. We assign different weights to different features in the IR matrix. In the next step, we carry out a multiplication operation to calculate the similarity of each two nodes according to the matrix IR . The significance of different features for the similarity calculation between graph nodes at layer l is captured by the parameter P . In the last step, we add a Relu activation function to improve the non-linear learning ability of the structure learning layer.

4.8. The merging layer and output layer

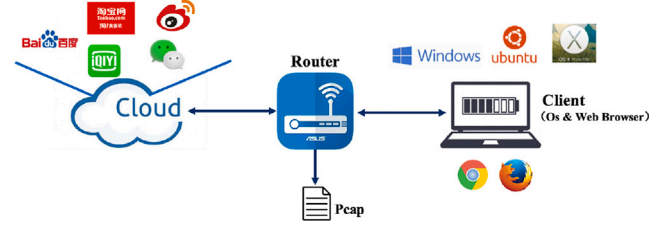
As we have demonstrated in Fig. 4, the neural network architecture repeats the graph convolution and pooling operations several times, thus we would observe multiple sub-graphs with different sizes at each level: $H^{(1)}, H^{(2)}, \dots, H^{(l)}$. For each sub-graph, we simply concat the result of mean-pooling and max-pooling along all nodes to generate the graph representation. The whole process is as follows:

$$r^l = \frac{1}{n^l} \sum_{p=1}^{n^l} H^{(l)}(p, :) \parallel \max_{p=1}^{n^l} H^{(l)}(p, :) \quad (3)$$

where $r^l \in \mathbb{R}^{2F}$. Finally, we sum up all graph representations at different levels together and feed the graph representation into MLP layer with a soft-max classifier. The feature vector $r \in \mathbb{R}^{2F}$ can be understood as extracted final fingerprint of the corresponding traffic flow. Then we take the application with the maximum probability as the prediction label.

$$r = r^1 + r^2 \dots + r^l \quad (4)$$

$$\hat{Y} = \text{softmax}(MLP(r))$$

**Fig. 6.** Data collection.

5. Experiments

5.1. Experimental settings

All experiments are compiled and tested on a Linux machine (CPU: Intel(R) Core(TM) i7, GPU: NVIDIA GeForce RTX 2060).

Dataset. To evaluate the performance of our proposed model, as shown in Fig. 6, we built a network environment and constructed 2 encrypted traffic dataset. The former is more inclined to verify the capabilities of classifying different types of HTTPS terminals, and the latter tends to verify the ability to identify web applications. When constructing the 2 dataset, we use Selenium WebDriver [33] to automatically control the communication between the client and the server to generate HTTPS traffic. Each session lasts 10 s. We collect around 1000 traffic flows of each type of web application over 30 days. Selenium WebDriver is a commonly used tool in Web automation test tasks. It can run directly in the browser to simulate the user's operations on the browser, such as opening the browser, logging in to the web page, and closing the browser. In addition, we use the packet capture module based on Scapy to passively collect traffic data passing through the router. To further evaluate the effectiveness of EC-GCN under Onion Router (Tor), we conduct experiments across a publicly-available dataset (ISCX-Tor). The details of the 3 datasets are as follows.

- OBW30. This dataset consists of 30+ thousands of encrypted traffic flows, which belong to 30 types of HTTPS (HTTP over SSL/TLS) traffic data after packet recombination and flow reduction techniques. As shown in Table 1, these traffic data are generated by 3 operating systems, 2 web browsers and 5 popular applications.
- HW19. This dataset consists of 19+ thousands of HTTPS traffic flows referring to 19 popular web applications after packet recombination and flow reduction techniques. As shown in Table 2, these applications include TikTok, Bilibili, Weibo, Github, Douyu, NeCmusic, Baidu, iCloud, QQ, Douban, etc.
- ISCX-Tor [34]. This dataset consists of 3021 encrypted traffic flows with 80 000 packets referring to 16 applications. This kind of traffic is more challenging for pattern extraction and classification as its ambiguous communications between the sender and the receiver through a distributed routing network. This dataset involves multiple encryption algorithms, including SSL/TLS, SSH etc.

Model configurations. In our experiments, the graph size of our model is set as 768 nodes, although the graph scale can be further expanded and to help improve the classification precision (We will discuss this configuration parameter in Fig. 7). The feature dimension of the graph representation, F , is set as 60. Considering the tradeoff of the precision and packet waiting time, we set M as 1000. 6 layers of graph convolutions and pooling operations are performed before the merging layer. The number of clusters is set as 60% of the number of nodes before applying the graph pooling layer. All models are trained for 100 epochs with early stopping applied when the validation loss starts to

Table 3
Experimental results on OBW30 dataset.

App	LSTM			DLWF			MAMPF			FS-Net			EC-GCN			App	LSTM			DLWF			MAMPF			FS-Net			EC-GCN		
	PR	RC	F1	PR	RC	F1	PR	RC	F1	PR	RC	F1	PR	RC	F1		PR	RC	F1	PR	RC	F1	PR	RC	F1	PR	RC	F1			
1	23.7	21.4	22.5	73.1	75.5	74.3	97.8	92.7	95.2	91.3	92.6	91.9	97.8	97.3	97.5	16	23.2	27.4	25.1	85.8	61.3	71.5	87.9	92.0	89.9	100	99.6	99.8	96.3	96.1	96.2
2	18.4	19.9	19.2	61.0	50.0	54.9	84.5	87.5	86.0	91.8	73.9	81.9	93.6	94.6	94.1	17	67.2	66.8	67.0	84.7	96.8	90.4	97.2	97.8	97.5	100	99.1	99.5	100	99.5	99.7
3	33.1	33.6	33.4	65.4	76.6	70.5	84.1	92.8	88.2	99.6	98.4	99.0	97.1	95.5	96.3	18	9.74	7.94	8.75	60.3	34.0	43.5	66.0	70.7	68.3	96.2	97.4	96.8	92.8	96.2	94.5
4	84.7	80.3	82.5	98.4	98.4	98.4	99.6	98.9	99.2	92.5	85.8	89.0	100	99.2	99.6	19	73.7	78.6	76.1	92.6	92.6	92.6	99.0	95.3	97.1	98.7	95.8	97.2	100	97.9	98.9
5	66.6	60.7	63.6	96.2	96.2	96.2	98.5	97.5	98.0	100	99.2	99.6	99.6	100	99.8	20	25.4	37.8	30.4	68.2	56.0	61.5	82.1	92.8	87.1	98.2	98.1	98.2	98.4	96.3	97.3
6	43.0	39.3	41.1	87.1	80.2	83.5	97.3	98.0	97.6	76.9	81.4	79.1	100	98.7	99.3	21	32.1	31.6	31.8	24.9	1.07	2.06	93.5	91.1	92.3	99.7	99.2	99.4	97.1	97.0	97.1
7	40.4	31.1	35.2	92.4	75.2	82.9	95.2	94.6	94.9	98.6	99.3	98.9	97.0	97.0	97.0	22	17.3	11.1	13.6	0	0	0	46.9	16.4	24.3	93.7	93.3	93.5	100	98.3	99.1
8	34.7	29.8	32.1	76.4	70.4	73.3	95.1	84.8	89.6	98.2	98.6	98.4	97.8	96.5	97.1	23	36.5	40.5	38.4	0	0	0	81.1	85.1	83.0	87.6	99.5	93.2	95.4	97.9	96.6
9	72.5	69.7	71.1	88.6	97.9	93.0	98.6	99.0	98.8	99.5	98.6	99.0	99.6	99.6	99.6	24	94.2	90.2	92.2	82.7	94.3	88.2	99.6	99.5	99.6	99.0	98.3	98.6	100	100	100
10	57.6	63.4	60.4	78.3	89.2	83.4	97.3	96.9	97.1	97.5	98.5	89.0	99.6	99.2	99.4	25	76.0	71.6	73.8	63.1	50.9	56.4	98.3	100	99.1	100	100	100	98.7	98.3	98.5
11	40.3	49.1	44.3	88.6	96.3	92.3	98.8	96.5	97.6	98.4	80.6	88.6	100	98.6	99.3	26	31.4	23.3	26.8	17.2	3.96	6.45	90.3	89.1	89.7	99.4	98.1	98.7	96.1	98.4	97.2
12	50.8	43.2	46.7	91.9	82.4	86.9	93.6	95.0	94.3	96.5	99.2	97.8	97.8	98.9	98.3	27	19.6	29.6	23.5	0	0	0	48.1	80.5	60.2	97.8	98.2	98.0	95.4	97.1	96.2
13	32.1	19.9	24.6	61.7	82.8	70.7	76.3	57.2	65.4	86.4	97.6	91.6	98.9	94.2	96.5	28	47.5	45.1	46.3	17.1	88.5	28.7	87.0	88.5	87.7	72.9	93.9	82.1	98.3	94.2	96.2
14	92.4	96.6	94.5	96.4	98.2	97.3	100	100	100	86.8	97.6	91.9	100	99.6	99.8	29	70.7	61.0	65.5	62.7	78.4	69.7	96.5	97.1	96.8	100	96.4	98.2	99.6	98.6	99.1
15	64.7	72.3	68.3	75.6	97.7	85.2	97.6	97.9	97.7	100	98.1	99.0	99.7	97.7	98.7	30	59.5	58.8	59.1	51.9	66.1	58.2	99.6	95.4	97.4	94.6	87.0	90.6	99.6	98.7	99.1
Avg	48.0	47.1	47.3	64.8	66.4	63.7	89.6	89.4	89.5	93.0	94.3	93.6	96.8	97.6	97.2																

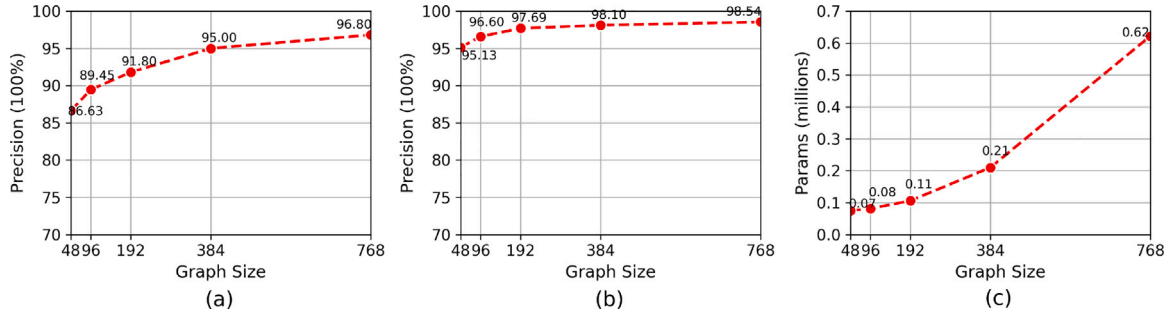


Fig. 7. (a-b) Precision vs Graph size on OBW30/HW19 dataset (c) Param vs Graph size.

drop. The batch size, learning rate and decay rate are set as 25, 0.001, 0.7 respectively. The model is trained using the Adam optimizer with default TensorFlow settings. The number of filters in each temporal and spatial convolution are all 8 and 60 respectively. The stride in each layer is set to 1.

Evaluation Metric and Baselines. In order to demonstrate the performance of our framework, we implement 4 representative baseline schemes, which also take the metadata of a traffic flow (such as the sequence of packet lengths/directions) as input and finish the encrypted traffic classification task. In this paper, we aim to propose an encrypted traffic classification method that is independent of different encryption protocols. So those studies combining features of the packet header and payload are not included in this experiment.

- Long short-term memory (LSTM) networks are a type of recurrent neural network capable of learning order dependence in sequence prediction/classification problems. In the experiments, we construct a network consisting of 2 LSTM layers to extract hidden features and a softmax classification layer as the output layer;
- Website Fingerprinting through Deep Learning (DLWF) [24] is a DL-based website fingerprinting method through the timing and sizes of network packets;
- MAMPF [22] is a method that uses the output probabilities of the message type and the length block Markov models as features to classify encrypted traffic with the random forest classifier;
- A Flow Sequence Network For Encrypted Traffic Classification (FS-Net) [27] is a BiGRU-based classification model that learns representative features of encrypted traffic flows and then classifies them.

We evaluate all the methods based on Precision (PR), Recall (RC) and F1-score (F1), where $PR = TP / (TP + FP)$, $RC = TP / (TP + FN)$ and $F1 = 2 * PR * RC / (PR + RC)$. TP, FP, FN and TN represent true positive, false positive, false negative and true negative respectively.

5.2. Experimental results

Basic Experiments. As shown in Table 3, EC-GCN achieves the best performance on the overall metrics and outperforms all the other methods on OBW30 dataset. Compared with other classification tasks, our experiment is more challenging. The traffic flows in this dataset all belong to HTTPS traffic. What we need to do is to accurately identify the application type, operating system type and browser type based on traffic characteristics at the same time. In addition, Considering encryption and the versatility of methods, we only use the metadata characteristics of the traffic data (packet length, direction) without extracting any characteristic in the payload or packet header. Our EC-GCN can obtain the best performance on all the overall metrics because it concurrently learns the temporal features and spatial graph features hidden in the network traffic metadata, while traditional methods can only capture one or two-order information of adjacent packets in the temporal dimension. The average precision of five models on

Table 4

Time consumptions of training on the dataset OBW30 and HW19.

Dataset	Time consumption (s)				
	Graph size				
	48	96	192	384	768
OBW30	166.3	250.4	459.5	1247.6	3173.6
HW19	112.7	202.8	370.4	925.4	2316.1

HW19 dataset are 0.9854(EC-GCN), 0.9535(FS-Net), 0.9109(MAMPF), 0.8103(DLWF) and 0.7589(LSTM) respectively. Therefore, our method can achieve up to 5%–10% precision improvement compared with the state-of-the-art methods and the improvements are seen for all application types.

Performance VS. Graph Size. We divide the value range of the packet length into a number of intervals, which corresponds to the graph size. We evaluate the impact of the graph size on the performance of our model from 3 aspects: precision, the number of parameters and time consumption of training. In Fig. 7(a-b), as the size of the graph increases, the classification precision of the model continues to improve, and will eventually increase to nearly 100%. As shown in Fig. 7(b), even when the graph size is small, our model can still achieve high classification performance, with the precision of 95.13% for a graph of 48 nodes. In addition, as shown in Fig. 7(c), the number of parameters of our model grows in proportion to the graph size. Even if the graph size is set to 768, the model parameters are far below the million level. The time consumption of training is shown in Table 4. Our model has a high training efficiency and only consumes hundreds of seconds with a graph size between 48–192. Overall, our model is lightweight. As we analyzed in Section 3, a small number of packet lengths covers most of the proportion. Therefore, the appropriate graph size can meet the actual classification requirements. Introducing GCN into our model also brings scalability and flexibility improvements, which allow us to choose different graph sizes for different application scenarios.

Performance VS. Attributes. We introduce the uplink/downlink information into the packet length sequence, and name the model EC-GCN-UD. The improvement from EC-GCN to EC-GCN-UD is not significant (e.g., 0.003 in Precision). A similar phenomenon happens in [27]. Generally, it is difficult to timely collect the uplink/downlink information of data packets when performing online classification tasks. The information in packet length sequences is rich enough to meet the actual classification requirements.

Robustness Test. We introduce two types of noise into the packet length sequence. The real-time packet length sequence of encrypted traffic data may be partially abnormal as a result of network failures. To examine the fault-tolerance ability in extreme environments, we randomly select a fraction of packet lengths in a traffic flow to sabotage their observations. We carry out two groups of studies and test the performance of different methods: (1) Packet loss test — We randomly drop 10% to 80% packet in each traffic flow; (2) Packet reordering test

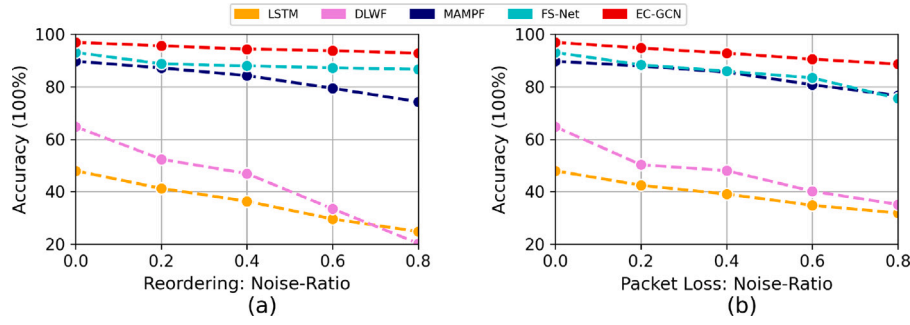


Fig. 8. Performance with different type of noise.

Table 5

Experimental results on ISCX-Tor dataset.

Performance	PR	RC	F1
LSTM	0.4382	0.4302	0.4270
DLWF	0.1553	0.1980	0.1644
MAMPF	0.6134	0.5783	0.5953
FS-Net	0.5080	0.5350	0.4590
EC-GCN	0.8351	0.8404	0.8214

Table 6

Experimental results of the ablation study.

OG-SL (Graph Learning)				✓	
DIFFPOOL (Graph Pooling)	✓				
Lightweight graph pooling		✓	✓	✓	
Graph structure learning				✓	
Precision	0.3535	0.5373	0.8743	0.8845	0.9686
Recall	0.6435	0.6789	0.8876	0.9033	0.9768
F1	0.4563	0.5998	0.8808	0.8938	0.9727

— We randomly select 10% to 80% packets in each traffic flow and disrupt their order. As shown in Fig. 8, each model suffering from varying degrees of degradation in the robustness test. Our model is shown to be more fault-tolerant with an average of 10–20% precision improvement compared with the state-of-the-art traffic classification models on the OBW30 dataset. Even when the fault ratio reaches 0.8, EC-GCN still has a strong classification capability. With the same amount of noise contamination, other models' performance drops dramatically without exception. We achieve similar results on HW19 dataset. From the perspective of multi-level graphs, noise contamination can only directly affect low-level graph representation. Our model can detect the changes of dependencies hidden in “contaminated” traffic samples and dynamically adjust the multi-level graph representations by the structure learning layer. In addition, making comprehensive use of time dependence and spatial dependence on graph structure further increases the fault-tolerance ability of our model.

Performance VS. Tor. As shown in Table 5, through learning the intrinsic relationship of packets and propagating information under multi-level graphs, our model can achieve great performance improvement over baselines. This task is more challenging as the traffic in Tor is not only multi-layer encrypted but also adversarially obfuscated. In addition, this publicly-available dataset is dominated by mice flows (about 27 packets in each flow on average). If adding some web applications with larger flow sizes, we can exploit more packets to enrich the propagated information under GCN and achieve better performance.

Ablation Study. In this paper, we propose a novel lightweight graph pooling layer and a novel graph structure learning layer to enable the down-sampling of the graph data and dynamically learn the weighted adjacency matrix respectively. We carry out an ablation study to demonstrate the effectiveness of our two novel modules on OBW30 dataset. We also implement 2 most widely used GCN modules for the performance reference: (1) OG-SL (CVPR 2019) [35] learn an optimal graph structure that best serves graph CNNs for semi-supervised

learning by integrating both graph learning and graph convolution in a unified network architecture. (2) DIFFPOOL (NIPS 2019) [32] propose a differentiable graph pooling module that can generate hierarchical representations of graphs and can be combined with various graph neural network architectures in an end-to-end fashion;

We replace the two novel modules in our network traffic classification framework with the above optional structures and compared the average results for all applications. As shown in Table 6, the highest performance is obtained when combining our two novel modules. First of all, the framework combined with our lightweight graph pooling layer achieves significant performance improvement than it with DIFFPOOL. Our novel graph pooling layer can help extract the exact fingerprint from encrypted traffic flows according to the graph representations at different levels. In addition, after further integrating with our novel graph learning layer, the accuracy of our framework is improved to 96.86%. On the contrary, the framework with the graph learning structures in OG-SL has little change in accuracy. We achieve similar comparison results on HW19 dataset. Our novel graph learning module can help define the similarity between two graph nodes from a global perspective, rather than just based on the similarity of features between two nodes. In summary, our two novel modules can help GCN understand the essence and capture inherent invariant characteristics in a flow.

6. Conclusion and future work

We propose a novel neural network (EC-GCN) to classify encrypted traffic. The experiment results demonstrate that our proposed neural network can achieve up to 5–20% improvement compared to other models. In addition, EC-GCN can be immune to specific types of packet length obfuscation operations (randomly packet loss and reordering) according to its performance in the robustness test. Given that only metadata characteristics are used as input, EC-GCN may be affected by some traffic shaping operations, such as padding incoming packets to regularize packet length sequences [36]. We can overcome this to some extent by integrating more metadata features, including packet type sequences, packet interval sequences and uplink/downlink sequences etc. The sequences composed of specific fields in consecutive packet headers can also help us to improve the robustness of EC-GCN.

In future work, we will extend our method to more application scenarios and further improve its performance. Investigation of robust algorithms that can cope with protocol changes and traffic obfuscation is also an important future direction.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The authors do not have permission to share data.

Acknowledgment

This work was supported in part by National Key R&D Program of China: 2022YFB3104800, and the National Natural Science Foundation of China No. 62102397.

References

- [1] K.D. Zeilenga, Internet Assigned Numbers Authority (IANA) Considerations for the Lightweight Directory Access Protocol (LDAP), RFC, 2006.
- [2] F. Constantinou, P. Mavrommatis, Identifying known and unknown peer-to-peer traffic, in: Fifth IEEE International Symposium on Network Computing and Applications, NCA 2006, 24–26 July 2006, IEEE Computer Society, Cambridge, Massachusetts, USA, 2006, pp. 93–102.
- [3] F. Risso, M. Baldi, O. Morandi, A. Baldini, P. Monclus, Lightweight, payload-based traffic classification: An experimental evaluation, in: ICC, 2008.
- [4] J. Yang, L. Jiang, Q. Tang, Q. Dai, J. Tan, PiDFA: A practical multi-stride regular expression matching engine based on FPGA, in: ICC, 2016.
- [5] H. Doroud, G. Aceto, W. de Donato, E.A. Jarchlo, A.M. López, C.D. Guerrero, A. Pescapè, Speeding-up DPI traffic classification with chaining, in: IEEE Global Communications Conference, GLOBECOM 2018, Abu Dhabi, United Arab Emirates, December 9–13, 2018, IEEE, 2018, pp. 1–6.
- [6] H. Yan, H. Li, M. Xiao, R. Dai, X. Zheng, X. Zhao, F. Li, PGSM-DPI: Precisely guided signature matching of deep packet inspection for traffic analysis, in: GLOBECOM, 2019.
- [7] R. Keralapura, A. Nucci, C. Chuah, Self-learning peer-to-peer traffic classifier, in: ICCCN, 2009.
- [8] T. Ongun, T. Sakharov, S. Boboila, A. Oprea, T. Eliassi-Rad, On designing machine learning models for malicious network traffic classification, 2019, CoRR.
- [9] R. Li, X. Xiao, S. Ni, H. Zheng, S. Xia, Byte segment neural network for network traffic classification, in: IWQoS, 2018.
- [10] M. Kim, A. Anpalagan, Tor traffic classification from raw packet header using convolutional neural network, in: ICKII, 2018.
- [11] T. van Ede, R. Bertolameotti, A. Continnella, J. Ren, D.J. Dubois, M. Lindorfer, D.R. Choffnes, M. van Steen, A. Peter, FlowPrint: Semi-supervised mobile-app fingerprinting on encrypted network traffic, in: 27th Annual Network and Distributed System Security Symposium, NDSS 2020, San Diego, California, USA, February 23–26, 2020, The Internet Society, 2020.
- [12] S. Yoon, J. Park, M. Kim, Signature maintenance for internet application traffic identification using header signatures, in: F.D. Turck, L.P. Gaspary, D. Medhi (Eds.), NOMS, 2012.
- [13] S. Lee, S. Yoon, M. Kim, Research on automatic header-signature naming system for internet service identification, in: APNOMS, 2015.
- [14] J. Yang, J. Narantuya, H. Lim, Bayesian neural network based encrypted traffic classification using initial handshake packets, in: IFIP, 2019.
- [15] M. Shen, Y. Liu, L. Zhu, K. Xu, X. Du, N. Guizani, Optimizing feature selection for efficient encrypted traffic classification: A systematic approach, IEEE Netw. (2020).
- [16] B. Anderson, S. Paul, D.A. McGrew, Deciphering malware's use of TLS (without decryption), J. Comput. Virol. Hack. Tech. (2018).
- [17] V.F. Taylor, R. Spolaor, M. Conti, I. Martinovic, Robust smartphone app identification via encrypted network traffic analysis, IEEE Trans. Inf. Forensics Secur. (2018).
- [18] H. Shi, H. Li, D. Zhang, C. Cheng, X. Cao, An efficient feature generation approach based on deep learning and feature selection techniques for traffic classification, Comput. Netw. (2018).
- [19] Y. Chen, T. Zang, Y. Zhang, Y. Zhou, Y. Wang, Rethinking encrypted traffic classification: A multi-attribute associated fingerprint approach, in: 27th IEEE International Conference on Network Protocols, ICNP 2019, Chicago, IL, USA, October 8–10, 2019, IEEE, 2019, pp. 1–11.
- [20] J. Cheng, R. He, Y. E, Y. Wu, J. You, T. Li, Real-time encrypted traffic classification via lightweight neural networks, in: IEEE Global Communications Conference, GLOBECOM 2020, Virtual Event, Taiwan, December 7–11, 2020, IEEE, 2020, pp. 1–6.
- [21] O. Aouedi, K. Piamrat, D. Bagadthey, A semi-supervised stacked autoencoder approach for network traffic classification, in: 28th IEEE International Conference on Network Protocols, ICNP 2020, Madrid, Spain, October 13–16, 2020, IEEE, 2020, pp. 1–6.
- [22] C. Liu, Z. Cao, G. Xiong, G. Gou, S. Yiu, L. He, MaMPF: Encrypted traffic classification based on multi-attribute Markov probability fingerprints, in: IWQoS, 2018.
- [23] Y. Fu, H. Xiong, X. Lu, J. Yang, C. Chen, Service usage classification with encrypted internet traffic in mobile messaging apps, IEEE Trans. Mob. Comput. (2016).
- [24] V. Rimmer, D. Preuveneers, M. Juárez, T. van Goethem, W. Joosen, Automated website fingerprinting through deep learning, in: NDSS, 2018.
- [25] V. Tong, H.A. Tran, S. Souihi, A. Mellouk, A novel QUIC traffic classifier based on convolutional neural networks, in: IEEE Global Communications Conference, GLOBECOM 2018, Abu Dhabi, United Arab Emirates, December 9–13, 2018, IEEE, 2018, pp. 1–6.
- [26] C. Dong, C. Zhang, Z. Lu, B. Liu, B. Jiang, CETAnalytics: Comprehensive effective traffic information analytics for encrypted traffic classification, Comput. Netw. 176 (2020) 107258.
- [27] C. Liu, L. He, G. Xiong, Z. Cao, Z. Li, FS-net: A flow sequence network for encrypted traffic classification, in: INFOCOM, 2019.
- [28] M. Lotfollahi, M.J. Siavoshani, R.S.H. Zade, M. Saberian, Deep packet: a novel approach for encrypted traffic classification using deep learning, Soft Comput. (2020).
- [29] A. Rasteh, F. Delpech, C.A. Melchor, R. Zimmer, S.B. Shouraki, T. Masquelier, Encrypted internet traffic classification using a supervised spiking neural network, 2021, CoRR.
- [30] X. Wang, S. Chen, J. Su, Automatic mobile app identification from encrypted traffic with hybrid neural networks, IEEE Access (2020).
- [31] M. Defferrard, X. Bresson, P. Vandergheynst, Convolutional neural networks on graphs with fast localized spectral filtering, in: D.D. Lee, M. Sugiyama, U. von Luxburg, I. Guyon, R. Garnett (Eds.), Advances in Neural Information Processing Systems 29: Annual Conference on Neural Information Processing Systems 2016, December 5–10, 2016, Barcelona, Spain, pp. 3837–3845.
- [32] Z. Ying, J. You, C. Morris, X. Ren, W.L. Hamilton, J. Leskovec, Hierarchical graph representation learning with differentiable pooling, in: NeurIPS, 2018.
- [33] M. Bures, M. Filipisky, SmartDriver: Extension of Selenium WebDriver to create more efficient automated tests, in: 6th International Conference on IT Convergence and Security, ICITCS 2016, Prague, Czech Republic, September 26, 2016, IEEE Computer Society, 2016, pp. 1–4.
- [34] A.H. Lashkari, G. Draper-Gil, M.S.I. Mamun, A.A. Ghorbani, Characterization of tor traffic using time based features, in: P. Mori, S. Furnell, O. Camp (Eds.), Proceedings of the 3rd International Conference on Information Systems Security and Privacy, ICISPP 2017, Porto, Portugal, February 19–21, 2017, SciTePress, 2017, pp. 253–262.
- [35] B. Jiang, Z. Zhang, D. Lin, J. Tang, B. Luo, Semi-supervised learning with graph learning-convolutional networks, in: CVPR, 2019.
- [36] R. Meier, V. Lenders, L. Vanbever, Ditto: WAN traffic obfuscation at line rate, in: 29th Annual Network and Distributed System Security Symposium, NDSS 2022, San Diego, California, USA, April 24–28, 2022, The Internet Society, 2022, URL <https://www.ndss-symposium.org/ndss-paper/auto-draft-195/>.



Zulong Diao received her Ph.D. degree in software engineering from Hunan University, Changsha, China, in 2019. He is currently an Associate Professor with the Network Technology Research Center, Institute of Computing Technology, Chinese Academy of Sciences, Beijing, China. His research interests include machine learning, traffic analysis and Internet measurement.



Gaogang Xie received the B.S. degree in physics, the M.S. degree, and the Ph.D. degree in computer science from Hunan University in 1996, 1999, and 2002, respectively. He is currently a Professor and the Director of the Computer Network Information Center, Chinese Academy of Sciences, Beijing, China. His research interests include Internet architecture, packet processing and forwarding, and Internet measurement.



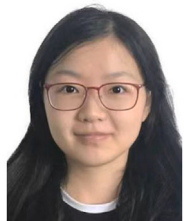
Xin Wang received the B.S. and M.S. degrees in telecommunications engineering and wireless communications engineering respectively from Beijing University of Posts and Telecommunications, Beijing, China in 1990 and 1993, and the Ph.D. degree in electrical and computer engineering from Columbia University, New York, NY.

She is currently an Associate Professor in the Department of Electrical and Computer Engineering of the State University of New York at Stony Brook, Stony Brook, NY. Before joining Stony Brook, she was a Member of Technical Staff in the area of mobile and wireless networking at Bell Labs Research, Lucent Technologies, New Jersey, and an Assistant Professor in the Department of Computer Science and Engineering of the State University of New

York at Buffalo, Buffalo, NY. Her research interests include algorithm and protocol design in wireless networks and communications, mobile and distributed computing, as well as networked sensing and detection. She has served in executive committee and technical committee of numerous conferences and funding review panels, and serves as the associate editor of IEEE Transactions on Mobile Computing. Dr. Wang achieved the NSF career award in 2005, and ONR challenge award in 2010.



Rui Ren is currently working toward his Ph.D. degree at the University of Chinese Academy of Sciences. His research interests include machine learning, Internet measurement, and AI Ops.



Xuying Meng received the B.S. degree from Wuhan University in 2013 and the Ph.D. degree from the University of Chinese Academy of Sciences in 2018. She is currently an Associate Professor with the Institute of Computing Technology, Chinese Academy of Sciences. She has published innovative works in top conference proceedings. Her current research interests include data mining and security protection of network services. She serves for numerous conference program committees.



Guangxing Zhang received the B.S. degree and the Ph.D. degree in computer science from Hunan University in 2002 and 2011, respectively. He is currently an Associate Professor with the Network Technology Research Center, Institute of Computing Technology, Chinese Academy of Sciences, Beijing, China. His research interests include wireless network and mobile computing, Internet architecture, packet processing and forwarding, and Internet measurement.



Kun Xie received her Ph.D. degree in computer application from Hunan University, Changsha, China, in 2007. She worked as a postdoctoral fellow in the department of computing in Hong Kong Polytechnic University from 2007.12 to 2010.2. She worked as a visiting researcher in the department of electrical and computer engineering in state university of New York at Stony Brook from 2012.9 to 2013.9.

She is currently a professor with Hunan University, Changsha, China. Her research interests include wireless network and mobile computing, network management and control, cloud computing and mobile cloud, and big data. She has published more than 70 papers in major journals and conference proceedings (including top journals IEEE/ACM TON, IEEE TMC, IEEE TWC, IEEE TC, and top conferences INFOCOM, ICDCS, SECON, IWQoS). She is a member of the IEEE.



Mingyu Qiao is currently working toward his M.S. degree at the University of Chinese Academy of Sciences. His research interests include machine learning, Internet measurement, and AI Ops.